Ders 4 Genişletilmiş Notu: Adapter, Bridge, Composite ve Decorator Kalıpları

Ders 4 Genişletilmiş Notu: Adapter, Bridge, Composite ve Decorator Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Yapısal Kalıplara Geçiş ve Adapter Kalıbı
- 2. Adapter Örneği: Çizim Programına Hazır Daire Sınıfı
- 3. Bridge Kalıbı
- 4. Bridge Örneği: İki Çizim Kütüphanesi
- 5. Composite Kalıbı
- 6. Composite Örneği: Makine ve Parça Kataloğu
- 7. Decorator Kalıbı
- 8. Decorator Örneği: Satış Makbuzu
- Örnek Sorular
- Dersin Toparlaması

1. Yapısal Kalıplara Geçiş ve Adapter Kalıbı

Geçen hafta yaratımsal (creational) kalıplar bitirildi. Bu derste yapısal (structural) kalıplara geçildi. İlk yapısal kalıp Adapter'dır. Hoca bunun nispeten basit bir kalıp olduğunu, öğrencilerin büyük olasılıkla daha önce farkında olmadan bu çözümü uyguladıklarını, ama literatürde ayrı bir kalıp olarak tanımlandığını bilmeyebileceklerini söyledi.

1.1 Amaç

Bir istekçi (client) sınıfımız ve bir hizmet veren sınıfımız vardır. Hizmet veren sınıftaki bir metodu çalıştırmak istiyoruz, ama istekçi başka bir metot adı ya da başka bir parametre dizilimi bekliyor. Üstelik istekçinin kaynak koduna müdahale edemiyoruz. Çözüm, araya istekçinin beklediği metot imzasına sahip, gövdesinde ise çağrıyı asıl işi yapacak nesneye yönlendiren bir adaptör sınıf koymaktır. Hocanın deyişiyle "olay bundan ibaret".

Kalıbın kullanılacağı an şöyle tanımlandı: elimizdeki bir sınıfın bir istekçi tarafından kullanılmasını önleyen tek engel, istekçinin gönderdiği mesajla sınıfın anladığı mesajların ad ve parametre diziliminin, yani imzalarının farklı olmasıdır.

1.2 Priz adaptörü benzetmesi

Hoca, İngiltere'deki priz standardını örnek verdi. Üç delikli bu prizde üst delik bir dil içerir; bu dil alttaki faz ve nötr deliklerini kapatır. İngiliz fişinin üst bacağı daha uzundur, önce o girer ve dili aşağı iterek alttaki kapakları açar. Avrupa standardındaki bir fişi takmak için önce bu dile bir şeyle bastırmak gerekir; ayrıca bacakların biçimi ve aralığı da farklıdır. Bu nedenle bir ülkenin standardını diğerine çeviren priz adaptörleri satılır. Yazılımdaki Adapter da aynı mantıkla çalışır.

1.3 Kalıp yapısı

Slayttaki yapı metne dökülünce (ekrandan, 00:16:30):

```
Client ---> Target (soyut)          Adaptee
            Request()               SpecificRequest()
               ^                        ^
               |                        | adaptee
            Adapter ---------------------
            Request()   // adaptee->SpecificRequest()
```

Uyarlanacak sınıfın (Adaptee) belirli bir metot imzası vardır; istekçi ise başka bir imza bekler. Adapter, `Request()` çağrısını alır ve `adaptee->SpecificRequest()` çağrısına yönlendirir. Hoca, üstteki `Target` sınıfının ve onun `Request()` metodunun çok da gerekli olmadığını, kalıptan çıkarılıp istekçinin doğrudan adaptörü kullanmasının hiçbir sakıncası olmadığını söyledi.

2. Adapter Örneği: Çizim Programına Hazır Daire Sınıfı

2.1 Problem

Bir çizim programı üzerinde çalışılıyor ve programın bir kısmı zaten gerçeklenmiş (ekrandan, 00:20:00). Bütün şekiller ortak bir soyut `Shape` üst sınıfında modellenmiş; somut alt sınıflar `Point`, `Line` ve `Square`dir. Bütün şekillerin konumu ayarlanıp okunabilir, ekrana çizilebilir, içi doldurulabilir, rengi değiştirilebilir ve ekrandan silinebilir:

```
Shape (soyut)
  +setLocation()
  +getLocation()
  +display()    {soyut}
  +fill()
  +setColor()
  +undisplay()  {soyut}
Alt sınıflar: Point, Line, Square (her biri display(), fill(), undisplay())
```

Hoca, şemada `setLocation()`'ın parametrelerinin gösterilmediğini, bir nokta için en azından `int x, int y` verilmesi gerektiğini belirtti. Bir doğru için başlangıç ve bitiş noktası (ya da başlangıç noktası, açı ve uzunluk), bir dikdörtgen için en ve boy gibi ek bilgiler alt sınıflara eklenmelidir. Bu örnek Dörtlü Çete'den değil, daha güncel bir kitaptan alınmıştır.

Sıra daire kodunu yazmaya gelmiştir. İki boyutlu grafiklerde daire çizmek kolay değildir: her piksel bir karecik olduğundan, karelerden oluşan bir alanda yuvarlak görünen bir daire çizmek için kenar yumuşatma (anti-aliasing) gerekir. Daha önce anti-aliasing dahil her şeyi yapan güzel bir daire sınıfı yazılmıştır, ama adı `XXCircle`dır ve metot imzaları farklıdır:

```
XXCircle
  +setLocation()
  +getLocation()
  +displayIt()
  +fillIt()
  +setItsColor()
  +undisplayIt()
```

Çerçeve `display()`, `fill()`, `setColor()` beklerken hazır sınıfta `displayIt()`, `fillIt()`, `setItsColor()` vardır. Hepsini baştan yazmaya ne vakit ne de istek vardır.

2.2 Çözüm

Çerçeveye dahil bir `Circle` sınıfı yazılır ama içinde iş mantığı kodlanmaz; bütün iş daha önce yazılmış `XXCircle` sınıfına yönlendirilir (ekrandan, 00:23:00):

```
class Circle extends Shape {
    ...
    private XXCircle myXXCircle;
    ...
```

```
    public Circle () {
        myXXCircle= new XXCircle();
    }

    public void display() {
        myXXCircle.displayIt();
    }
    ...
}
```

`Circle`, uydurulacak sınıf türünden bir üye alana sahiptir ve bütün metotları yalnız ilgili çağrıyı bu üye üzerinden yürütür. Bir değer döndürülecekse o da `return myXXCircle...` biçiminde iletilir. Bu örnekte istekçi `Shape` (çerçeve), uyarlanacak sınıf `XXCircle`, adaptör ise `Circle`dır. Slayttaki şemada `Circle` ile `XXCircle` arasında içi dolu elmaslı bir parça-bütün ilişkisi vardır.

> **Ders sonrası not:** Slayttaki kodda metot başlığı `void public display()` diye yazılmıştır. Java'da erişim belirleyici dönüş türünden önce gelir; doğrusu `public void display()`dir.

3. Bridge Kalıbı

3.1 Amaç

Bridge (köprü) kalıbının amacı, bir nesnenin arayüzünü (soyutlamasını) gerçekleme ayrıntılarından ayırmaktır. Hoca, eskiden Win32 kodu yazanların bunu "handle" olarak bildiğini hatırlattı: Windows programa bir handle verir ve program işlerini o handle üzerinden yapar.

3.2 Dörtlü Çete örneği: farklı masaüstü ortamlarında pencere

Bir GUI çerçeve programında pencere soyutlaması yapılıyor ve bu çerçeve farklı masaüstü ortamlarında kullanılmak isteniyor. İlk akla gelen, soyut bir pencere sınıfından kalıtımla her ortam için ayrı pencere sınıfları türetmektir: `XWindow`, `PMWindow` (Presentation Manager), bugün için `GNOMEWindow`, `KDEWindow`. Hoca, bu ilk düşünceyle problemi Abstract Factory'ye benzetmenin mümkün olduğunu, ama önemli farkları olduğunu söyledi.

Sonra yeni bir istek gelir: pencerenin görev çubuğunda simge haline getirilmiş (iconified) biçimi de desteklenecektir. `IconWindow`, `Window` ile epey ortak kod paylaşır ama farklılaşan noktaları da vardır. Simge pencereye özel kod yazıldığı için onun da soyut olması ve her ortam için somut alt sınıfları (`XIconWindow`, `PMIconWindow`) olması gerekir. Ardından GNOME desteği istenirse hem pencere hem simge pencere tarafına birer kardeş sınıf, yani iki sınıf eklemek gerekir. Şeffaf pencere (`TransparentWindow`) istenirse yine bir soyut sınıf ve her ortam için birer alt sınıf eklenir. Hocanın deyişiyle işler "dallanıp budaklanır".

Hoca burada Abstract Factory ile farkı vurguladı: Abstract Factory'de farklı pencere yöneticilerindeki düğme, radio button ve checkbox gibi ürünler birbirinden çok farklı şeylerdir. Pencere, simge pencere ve şeffaf pencere ise ortak kodun çoğunu paylaşır. Bu yüzden bu problem için Abstract Factory uygun değildir, daha akıllıca bir şey gerekir.

3.3 Dörtlü Çete'nin çözümü

Pencere soyutlaması ve pencere gerçeklemeleri ayrı sınıf hiyerarşilerine konur ve bu iki hiyerarşi bir köprü üzerinden birleştirilir (ekrandan, 00:30:00):

```
Window (soyut) ----imp----> WindowImp (soyut)
  DrawText()                  DevDrawText()   {soyut}
  DrawRect()                  DevDrawLine()   {soyut}
  DrawRect() notu:
    imp->DevDrawLine()
    imp->DevDrawLine()
    imp->DevDrawLine()
    imp->DevDrawLine()
```

```
Window alt sınıfları:              WindowImp alt sınıfları:
  IconWindow                         XWindowImp
    DrawBorder()  // DrawRect()        DevDrawText()  // XDrawString()
                  // DrawText()        DevDrawLine()  // XDrawLine()
  TransientWindow                    PMWindowImp
    DrawCloseBox() // DrawRect()       DevDrawLine()
                                       DevDrawText()
```

`Window` ile `WindowImp` arasında `imp` (implementation kısaltması) adlı tek bir üye alan vardır. Şemada parça-bütün gibi görünse de aslında birebir sahipliktir. Bir pencerenin sınırını çizmek için bir dikdörtgen, yani dört çizgi çizilmesi gerekir; `DrawRect()` bunu dört kez `imp->DevDrawLine()` çağırarak yapar. Hoca, `Dev` önekinin büyük olasılıkla "device" anlamına geldiğini söyledi.

Örneğin simge pencere sınırını çizerken hem dikdörtgen çizer hem de metnini yazar; şeffaf bir pencere ise başlığını yazmayabilir, yalnız dikdörtgen çizer. Bu farklar soyutlama tarafında kodlanır. Çizgi çizme ve metin yazma gibi sistem çağrıları ise gerçekleme tarafındadır: `XWindowImp` metin yazmak istediğinde X'in metin çizme fonksiyonunu, `PMWindowImp` ise kendi pencere yöneticisinin kütüphanesini kullanır.

Varsayılan davranış olarak normal pencere düşünülürse `Window` somut da yapılabilir, ya da `IconWindow`'un kardeşi olarak bir `RegularWindow` sınıfı eklenebilir. Bu tasarımda geçici pencere (`TransientWindow`) eklemek için yalnız bir sınıf yazılır; onun X ya da PM gerçeklemesi için başka yerlere sınıf eklemek gerekmez. KDE desteklenmek istenirse yalnız bir `KDEWindowImp` sınıfı eklenir. Pencere nesneleri ilklendirilirken köprüyü sağlayacak gerçekleme nesnesi verilir.

Hoca, Bridge'in Adapter gibi hemen akla gelen bir kalıp olmadığını, keşfetmenin daha ayrıntılı bir düşünce ve deneyim gerektirdiğini belirtti.

3.4 Kalıp yapısı ve kullanım

Kalıp yapısında bir soyutlama (`Abstraction`) ve onu gerçekleyebilecek somut sınıflar için bir üst soyut sınıf (`Implementor`) vardır. Abstraction'a bir Implementor verilir ve Abstraction'ın bütün işlemleri isteği Implementor'a yönlendirir. Öte yandan Abstraction'dan ek özelliklere sahip somut alt sınıflar (`RefinedAbstraction`) kolayca tanımlanabilir. Ona hangi somut gerçekleyici (`ConcreteImplementor`) verilirse iş o gerçekleyici üzerinden yürür; RefinedAbstraction hangi somut gerçekleyiciyle çalıştığını bilmek zorunda değildir.

Kalıp şu durumlarda kullanılabilir:

- Soyutlama ile gerçeklemeleri arasındaki bağın kalıcı olması istenmiyorsa ya da işi karmaşıklaştırıyorsa.
- Hem soyutlamanın hem gerçeklemelerin ayrı ayrı özelleşme hiyerarşilerine tabi tutulması isteniyorsa.

4. Bridge Örneği: İki Çizim Kütüphanesi

4.1 Problem

Yine bir çizim programı üzerinde çalışılıyor. İlk aşamada yalnız dikdörtgenler gerçeklenecek, zamanla başka şekiller de eklenecek. Mesele şudur: iki farklı dikdörtgen ailesi vardır ve farklı aileden dikdörtgenler farklı yoldan çizilir. Bir dikdörtgen dört köşeyle tanımlanır, ama üst köşelerin y koordinatları aynı olduğu için `x1, y1, x2, y2` gibi dört tam sayı yeterlidir. Dikdörtgen çizilirken dört çizgi çizilir ve bu iş için iki hazır kütüphane vardır (ekrandan, 00:40:00):

- `DP1`: `draw_a_line()` metodu çizginin başlangıç ve bitiş koordinatlarını `x1, y1, x2, y2` sırasıyla alır.
- `DP2`: `drawline()` metodu koordinatları `x1, x2, y1, y2` sırasıyla alır.

Daha önce bu iki biçimde hazırlanmış bir dolu çizim dosyası da vardır; her dosya biçimindeki koordinat sıralaması kendi kütüphanesine uygundur. "İki aile, iki kütüphane" kastedilen budur.

4.2 İlk çözüm: kalıtımla iki dikdörtgen

"Neyin değiştiğini bul ve onu sarmala" ilkesinden yola çıkarak soyut bir `Rectangle` sınıfı ve ondan türeyen `V1Rectangle`, `V2Rectangle` yazılır (ekrandan, 00:40:00):

```
Client ---> Rectangle (soyut)
              +draw()
              #drawLine()   {soyut}
            V1Rectangle: #drawLine()  ...> DP1 (+draw_a_line())
            V2Rectangle: #drawLine()  ...> DP2 (+drawline())
```

`draw()` gövdesi kodludur ve `drawLine()`'ı çağırarak dikdörtgeni çizer. `V1Rectangle.drawLine()` DP1'in, `V2Rectangle.drawLine()` DP2'nin metodunu çağırır. Slayttaki açıklama `drawLine`'ın neden protected olduğunu anlatır:

- `drawLine` metodu nesnenin iç çalışmasıyla ilgilidir, bu yüzden public yapılmamalıdır.
- Private metotlar kalıtımla aktarılmaz.
- Protected metotlar hem farklı türden nesnelere görünmez hem de alt sınıflara kalıtımla aktarılır.

  Ders sonrası not: Hoca derste bir ara private metotların kalıtımla "aktarıldığını ama override edilemediğini" ya da "aktarılıp doğrudan kullanılamadığını" söyledi. Java dil tanımına göre private üyeler alt sınıfa kalıtımla aktarılmış sayılmaz ve override edilemez; alt sınıfta aynı imzayla yazılan bir metot ayrı, ilgisiz bir metottur. Hocanın "aktarılıyor" derken büyük olasılıkla alanların nesnenin içinde bellekte yer almasını kastettiği düşünülebilir. Slayttaki "private metotlar kalıtım ile aktarılmaz" ifadesi doğrudur.

4.3 Yeni gereksinim: daireler

Beklendiği gibi yeni bir gereksinim gelir: iki farklı dikdörtgen ailesi gibi iki farklı daire ailesi, sonra iki farklı üçgen ailesi olacaktır. Birinci dikdörtgen ailesi ile birinci daire ailesi ortaktır, çünkü ikisi de DP1 ile çizilir; ikinci aileler de DP2 ile çizilir. Çizim programının bir şeklin dikdörtgen mi daire mi olduğunu bilmesine gerek olmadığı için ortak bir soyut `Shape` üst sınıfı eklenir. `Rectangle`'ın kardeşi olarak soyut bir `Circle` sınıfı gelir; onun da `V1Circle` (DP1'in daire çizme metodunu çağırır) ve `V2Circle` (DP2'ninkini çağırır) alt sınıfları olur.

Bu yolla sınıf sayısı hızla artar. Şu anda iki şekil (dikdörtgen ve daire) ve iki aile vardır; $2 \times 2 = 4$ somut şekil sınıfı gerekir. Üçüncü bir şekil eklenirse $2 \times 3 = 6$ olur; yeni bir aile eklenirse yine çarpanlardan biri artar. Hoca, bu yolun yanlış olduğunu erkenden fark etmenin daha iyi olacağını söyledi.

4.4 Bridge ile çözüm

Sorunun kök nedeni, farklı şekil türleri ile farklı çizim yollarının birbirinden net ayrılamaması, yani aralarındaki sıkı bağlaşımdır. Çeşitli şekil soyutlamaları ve bunların farklı gerçeklenme (çizilme) ayrıntıları vardır. Bunlar ayrı ayrı özelleşirse aradaki çapraz bağlar kesilir ve sınıf sayısı azalır. Bu da akla Bridge'i getirir (ekrandan, 00:50:00):

```
Shape Abstraction                    Drawing Implementation
Shape (soyut) ------_dp------------> Drawing (soyut)
  +draw()        {soyut}               +drawLine()
  #drawLine()                          +drawCircle()
  #drawCircle()                      V1Drawing: +drawLine(), +drawCircle() ...> DP1 (+draw_a_l
Rectangle: +draw()                   V2Drawing: +drawLine(), +drawCircle() ...> DP2 (+drawline
Circle:    +draw()
```

Şekiller bir kalıtım hiyerarşisine, çizim işlemleri ayrı bir kalıtım hiyerarşisine konur. Soyut `Shape`'e, kendisi de soyut olan `Drawing`'in alt sınıflarından somut bir nesne verilerek hangi çizim yönteminin kullanılacağı belirlenir. Üçgen eklenecekse yalnız `Shape`'in altına bir `Triangle` eklenir. Üçgen `drawLine` ile çizilebilir; çizilemese bile eski şekiller zaten DP1 ya da DP2 ile çizilebildiğinden bu kütüphanelerde gereken komut vardır. Yeni şekil yeni bir çizim yöntemi gerektirirse yalnız o yöntem DP1 ve DP2'ye (ve `Drawing` hiyerarşisine) eklenir; yine de yeni

bir sınıf ortaya çıkmaz. Böylece her yeni şekil için yalnız bir sınıf yazılır ve kopyala-yapıştır programlama yapılmaz.

Kod (ekrandan, 00:51:10):

```
package bridge.solution2;
public abstract class Shape {
    public abstract void draw( );
    private Drawing _dp;
    public Shape (Drawing dp) { _dp= dp; }
    protected void drawLine(double x1, double y1,
        double x2, double y2) { _dp.drawLine(x1, y1, x2, y2); }
    protected void drawCircle(double x,double y,
        double r ) {_dp.drawCircle(x, y, r); }
}
public class Circle extends Shape {
    private double _x, _y, _r;
    public Circle (Drawing dp,
                   double x,double y,double r) {
        super( dp) ;
        _x= x; _y= y; _r= r ;
    }
    public void draw () { drawCircle(_x,_y,_r); }
}
```

Şeklin kurucusuna ilgili çizim gerçeklemesi mutlaka verilir. `Circle` kendini çizerken üst sınıftan aldığı `drawCircle`'ı kullanır; `drawCircle` da şeklin ilişkili olduğu çizim gerçeklemesi üzerinden işi yapar. `Circle` bu metodu override etmez, yalnız kullanır. Dairede yalnız merkez koordinatları ve yarıçap vardır; `Rectangle` da kendi üye alanlarıyla benzer biçimde yazılır. Alt sınıfın hangi gerçekleme üzerinden çizileceği konusunda hiçbir endişe duymasına gerek yoktur. `V1Drawing` DP1'i, `V2Drawing` DP2'yi kullanır. Hoca, kodda DP1 ve DP2 metotlarının static olduğunu, sınıf adı üzerinden çağrıldığını ve gerekirse kütüphaneye yeni bir static çizim metodu eklenebileceğini gösterdi.

4.5 Bu örnekten çıkarılan dersler

Hoca, "neyin değiştiğini bul ve sarmala" ilkesinin geçerli olduğunu, ama birden fazla şey değişiyorsa kalıtım ilişkisini doğru kurmaya çok dikkat etmek gerektiğini vurguladı. Aksi halde ilk akla gelen çözümde olduğu gibi çok fazla sınıf ortaya çıkar. Bridge ile beklenen değişiklikler, sınıf sayısını hızla artırmadan, yalnız bir alt sınıf eklenerek karşılanır.

4.6 Protected yerine public final

Moladan sonra hoca, `drawLine` ve `drawCircle`'ı protected yapmanın hiçbir zorunluluğu olmadığını söyledi. Alt sınıflardan çağrılacaklarsa public de olabilirler. `Shape` soyut bir sınıf olduğundan ondan nesne oluşturulup bu metotlar dışarıdan çağrılamaz; metotlar static olmadığından `Shape.drawLine` diye de çağrılamaz. Hoca bu nedenle protected'ın burada ne kazandırdığını göremediğini söyledi. Asıl istenen alt sınıfların bu metotları override edip köprü üzerinden yapılan çizimi bozmaması ise, metot `public final` yapılabilir.

Bunu Eclipse'te `dp.bridge.solution3` adlı yeni bir pakette denedi (ekrandan, 01:12:40):

```
package dp.bridge.solution3;

public abstract class Shape {
    public abstract void draw( );
    private Drawing _dp;
    public Shape (Drawing dp) { _dp= dp; }
    public final void drawLine(double x1, double y1,
            double x2, double y2) { _dp.drawLine(x1, y1, x2, y2); }
    protected void drawCircle(double x,double y,
```

```
            double r ) {_dp.drawCircle(x, y, r); }
}
```

Karede `drawCircle` henüz protected'tır; hoca ikisini de `public final` yaptığını ve kodun geri kalanında hiçbir hata çıkmadığını söyledi. Protected'ın gerçekten gerekli olduğu yerler vardır, ama burası onlardan biri değildir.

5. Composite Kalıbı

5.1 Amaç ve örnek

Composite, "bileşenlerden oluşmuş" demektir; hoca, kalıbın ne işe yaradığının adından anlaşılabileceğini söyledi. Amacı hiyerarşik parça-bütün ilişkilerini desteklemektir. Bu kalıp sayesinde tek tek nesneler ve nesne toplulukları istekçiye tamamen aynı biçimde görünür.

Dörtlü Çete'nin örneği bir grafik çizim programıdır. Programdaki grafik bileşenler (çizgiler, dörtgenler, metinler) `Draw` metotları kodlanırsa kendilerini çizebilir. Bu bileşenler bir resim altında gruplanabilir; bir resim alt resimlerden de oluşabilir. Hoca bunu sunum programında gösterdi: birkaç şekil ekleyip bazılarını bir grup, bazılarını başka bir grup yaptı, sonra bu iki grubu birlikte gruplandırdı. Grubu çözdüğünde bütün bileşenler tek tek ayrılmadı, iki alt grup olarak kaldı. Composite bu davranışı modeller.

Kodlamada çizgi, dikdörtgen, metin gibi bütün şekiller `Graphic` adlı soyut bir üst sınıfta toplanır. Bir resim (`Picture`) bir ya da daha fazla grafik bileşen içerebildiğinden o da `Graphic`'in alt sınıfıdır. `Picture`'ın içinde `Graphic` türünden nesnelerin listesi vardır; bunlar somut `Line`, `Rectangle`, `Text` ya da başka bir `Picture` olabilir. `Picture.Draw()` listedeki her elemanın `Draw()` metodunu çağırır; böylece hem alt resimler hem de alt resimlerin içindeki bileşenler sırayla çizilir. Belirli bir alt bileşene erişmek için bir getter ve parça-bütün ilişkisini yöneten başka metotlar da eklenir.

5.2 Kalıp yapısı

Slayttaki yapı (ekrandan, 01:20:00):

```
Client ---> Component (soyut)  <-----------------+
              Operation()                        |
              Add(Component)                     | children
              Remove(Component)                  |
              GetChild(int)                      |
            Leaf                      Composite --+
              Operation()               Operation()   // forall g in children g.Operation();
                                        Add(Component)
                                        Remove(Component)
                                        GetChild(int)
```

```
Nesne yapısı:
aComposite
 +-- aLeaf
 +-- aLeaf
 +-- aComposite
 |    +-- aLeaf
 |    +-- aLeaf
 |    +-- aLeaf
 +-- aLeaf
```

Bir işlem yapılacağı zaman, işlemin bütün alt bileşenlerde yürütülmesi için Composite'in işleminde bir "forall" döngüsü bulunur. Bileşenler arasında ortaya bir ağaç yapısı çıkar. Bütünü simgeleyen kök düğüm mutlaka bir composite'tir. Uç düğümlere yaprak (leaf), aradaki düğümlere ara düğüm denir; ara düğümler de composite'tir.

Slayttaki kalıp katılımcıları:

- **Component:** Nesneler topluluğunun katılımcıları için ortak kullanım arayüzü sunar. Bir bütünün parçaları için erişim ve düzenleme metotları (`Add`, `Remove`, `GetChild`) tanımlar.
- **Leaf:** Topluluktaki parçaları simgeler. Hiyerarşinin en altındaki düğümlerdir, çocukları olamaz.
- **Composite:** Hiyerarşik yapının ara düğümleridir. Farklı tür ara düğümler varsa bu sınıfın alt sınıfları biçiminde modellenir.
- **Client:** Parça ve toplulukları benzer biçimde kullanabilir. Client bir Visitor olabilir.

Hoca, farklı türde ara düğümler gerekiyorsa bunların `Composite`'in kardeşi olarak modellenmesinin sakıncalı olduğunu söyledi: `Add`, `Remove`, `GetChild` gövdelerinin orada da kodlanması gerekir ve kod kopyalanmış olur. En doğrusu farklı tür composite'leri `Composite`'in alt sınıfları olarak modellemektir.

Client yapının kökünü biliyorsa, kökte çağrılan işlem bütün ara düğümlere yayılır. Visitor kalıbı henüz görülmedi, ama bütün düğümler arasında dolaşarak kökün işleminden farklı, ayrıntılı işlemler yapmak gerekiyorsa Client bir Visitor olabilir.

5.3 Ne zaman kullanılır

- Nesneler arasında hiyerarşik parça-bütün ilişkilerinin modellenmesi gerekiyorsa. Hoca örnek olarak orduyu verdi: ordunun içinde kolordular, onların içinde tümenler, tugaylar, alaylar.
- Nesneler topluluğuna ve topluluktaki tek tek bileşenlere aynı biçimde erişim gerekiyorsa.

5.4 Karar verilmesi gereken ayrıntılar

Alt düğümün üst düğümü bilmesi. İstenirse alt düğüm, üst düğümünden haberdar kılınabilir. Hoca slayttaki ağaç üzerinde bir yaprağın ve bir ara düğümün kendi üst düğümünü bilmesini ayrı ayrı (yeşille) çizdi, sonra bunun yerine `Component` düzeyinde tek bir başvuru (turuncuyla) önerdi (ekrandan, 01:30:00): her bileşenin bir üst bileşeni olacaktır; kök düğümde bu başvuru `null` olur. Böylece iki ayrı ilişki tek bir `myParentComponent` üyesiyle modellenir.

Bu başvuru sayesinde Client'ın kök composite'i bilmesi gerekmez. Herhangi bir yaprağa bir işaretçisi varsa, onun üst düğümünü sorarak köke kadar çıkabilir. Hoca bunun için bir `getRoot()` metodu tarif etti: üst düğüm başvurusu null değilse üst düğümün `getRoot()` sonucunu döndürür; null ise kök kendisidir ve `return this` der. Bu metot özyinelemeli (recursive) olarak köke kadar gider.

Düğüm silme. Yaprak silmek sorun değildir. Ara düğüm silinmek istendiğinde ise, silinmesi istenmeyen yaprakları olabilir. Seçenekler şunlardır:

- İş mantığı uygunsa, ara düğüm silinince bütün alt ağaç da silinir.
- Çocuğu olan composite'lerin silinmesi engellenir ya da kullanıcıya "emin misiniz" diye sorulur.
- Silinen composite'in çocukları, onun üst düğümüne aktarılır.

Çocuğun ebeveyninden haberdar olması, ters yönlü başvuruları kurmak ve korumak için biraz ek kod gerektirse de, bu tür silme işlemlerini basitleştirir: düğüm kendini silerken çocuklarını `Add` metotlarıyla kendi üst düğümüne ekler. Silme metodunun nereye konacağı da bir karardır: yapraklara değil, büyük olasılıkla composite'e; kökte değilse isteği köke iletme ya da üst düğüm biliniyorsa doğrudan çocukları aktarma gibi seçenekler vardır. Hoca, iç mantık neyi gerektiriyorsa kararın ona göre verileceğini söyledi.

5.5 Kalıbın zayıf yönü

Kalıpta kalıtım ilişkisinin özüne aykırı bir durum vardır: `Leaf` bir alt sınıftır, ama üst sınıftaki çocuk yönetim metotları kalıtımla ona da geçer. `Add`, `Remove`, `GetChild` composite için anlamlıdır ve gövdeleri orada kodlanır; yaprağın ise çocuğu olamaz, bu metotlar onun için anlamsızdır. Eş biçimde kullanım kolaylığı için soyut `Component` sınıfı eklendiğinden bu metotlar yaprakta da yazılmak zorundadır; aksi halde derleme hatası alınır. Yaprakta bu metotların gövdesi boş bırakılır. Metot boolean döndürüyorsa iç mantığa göre seçim yapılır: "ekleme başarılı mı" sorusuna yaprak `return false` diyebilir ya da şikayet etmeden geçmesi istenirse `return true`. `GetChild(int)` yaprakta ne parametre verilirse verilsin `return null` döndürür.

> **Hocanın vurgusu:** Bu metotlar Leaf sınıfında hiç güzel durmayacak, ama bu kalıbın getirdiği esneklik karşılığında ödenen bir bedeldir. Çok büyük bir bedel değil, ama rahatsız etmeye yetecek kadar bir bedeldir.

6. Composite Örneği: Makine ve Parça Kataloğu

6.1 Problem

Slayttaki problem (ekrandan, 01:40:00):

- Makineler ve parçalarından oluşan bir üretim kataloğumuz var.
- Bu katalogda makinelerin ve/veya parçaların hangi fabrikada üretildiği, maliyeti gibi bilgilerden çeşitli raporlar üretilmesi isteniyor.
- Bazı makineler kendi içinde makine parçaları içerebilir.
- Bu durumda Composite kalıbını uygulamak yerinde olacaktır.

Slayttaki esin kaynağı "DP Java Workbook"tur; hoca örneği biraz değiştirdiğini söyledi.

6.2 Tasarım

Makine de makine parçası da aynı biçimde erişilmek istendiğinden soyut bir `ManufacturingItem` sınıfı oluşturulur; kalıp da zaten bunu gerektirir. Bütün ortak bilgiler bu soyut sınıfta toplanır:

- `machineID`: makine için makine kimliği, parça için aslında parça kimliğidir. Hoca, parçayı da bir makine gibi düşünebileceğimiz için adın çok anlamsız olmadığını, ama isteyenin `itemID` diyebileceğini söyledi.
- `name` ve `factory`: adı ve hangi fabrikada üretildiği. Örneği karmaşıklaştırmamak için ikisi de `String` olarak modellenmiştir. Hoca, özellikle `factory` için bunun modelleme açısından doğru olmadığını, ayrı bir `Factory` sınıfı yazılıp `ManufacturingItem` ile istenen türde ilişki kurulması gerektiğini, bunun ileride görülecek kod kusurları arasında olduğunu söyledi.
- Bu üyeler için getter ve setter metotları.

`MachinePart`, içinde bulunduğu makinenin kimliğini de bilmelidir; bu yüzden `parentID` üyesi ve getter-setter'ı vardır. Hoca, `parentID`'yi üst sınıfa koymadığını, çünkü gereksinimlerde makinelerin alt makineleri olacağı söylenmediğini, yalnız parçalar için anlamlı olduğunu açıkladı. Kalıbın genel zayıflığına her noktada düşmek gerekmez. Ancak hoca örneği hazırlarken makinelerin alt makineleri de olabileceğini düşündüğünü ve bunu derste sözlü olarak söylediğini hatırladı. Makinenin alt makineleri olabilecekse `parentID` üst sınıfa (`ManufacturingItem`'a) taşınmalıdır; hoca bunu slayta açıklama olarak ekledi.

`Machine` sınıfında parça listesi, `addPart`, `removePart` ve alt bileşenler arasında dolaşmayı sağlayan bir iterator döndüren `getParts` vardır. Bu metotlar üst sınıfta soyuttur; `MachinePart`'ta `getParts return null` der, `addPart` ve `removePart` ise `true` ya da `false` döndürür. `Machine`'de `parentID` üyesi yoktur; `getParentID` sıfır (ya da sıfır anlamlıysa eksi bir) döndürür, `setParentID`'nin gövdesi boştur. `MachinePart`'ın kurucusu üst sınıfın kurucusunu çağırır ve ek olarak `parentID`'yi alır.

6.3 Aynı parçanın birden fazla makinede bulunması

Bir parça yalnız tek bir makinenin içinde bulunuyorsa, parçanın üst düğümü onu hangi makineye eklediysek odur. Ama aynı parça birden fazla makineye ekleniyorsa, o parçayı tek bir `MachinePart` nesnesiyle modellemek doğru olmaz. Çünkü `addPart` metodu bir parçayı eklerken parçanın `parentID`'sinin üzerine kendi kimliğini yazar. Bu durumda örneğin "dişli" için her makineye ayrı bir nesne oluşturulmalı ve `parentID`'leri farklı olmalıdır; nesneleri birbirinden ayırt etmek için adlarına hangi makinenin dişlisi olduklarını yazmak yararlı olabilir.

6.4 Rapor sınıfı

`Report` sınıfına basitçe bir static metot eklenmiştir: belirli bir fabrikada üretilen öğeleri bulan `findItems`. Makinenin maliyetini hesaplamak gibi raporlar Visitor kalıbına bırakılmıştır. Katalog bir `ManufacturingItem` dizisinde tutulur. Kod (ekrandan, 01:48:00):

```
public class Report {
    public static int findItems(ManufacturingItem items[],String factory) {
        int nr = 0;
        System.out.println("Items manufactured in " + factory + ":");
        System.out.println("Nr.\tID\tName");
        System.out.println("---------------------------------");
        for( ManufacturingItem item : items ) {
            if( item==null )continue;
            if( item.getFactory().compareToIgnoreCase(factory) == 0 ) {
                nr++; System.out.println( nr + ".\t" + item.getMachineID()
                        + "\t" + item.getName() );
            }
            Iterator<ManufacturingItem> subItems = item.getParts();
            if( subItems==null ) continue;
            while( subItems.hasNext() ) {
                ManufacturingItem part = subItems.next();
                if( part.getFactory().compareToIgnoreCase(factory) == 0 ) {
                    nr++; System.out.println( nr + ".\t" +
                            part.getMachineID() + "\t" + part.getName() );
                }
            }
        }
        if( nr == 0 )       System.out.println("No such item found.");
        return nr;
    }
}
```

Önce bir başlık satırı yazılır. Dizi seyrek olabileceği için null elemanlar atlanır. Öğenin fabrikası, parametre olarak verilen adla büyük-küçük harf ayrımı yapılmadan karşılaştırılır; hoca `compareToIgnoreCase` yerine `equalsIgnoreCase` de kullanılabileceğini söyledi. Eşleşirse sayaç bir artırılır ve sıra numarası, kimlik ve ad yazılır. Makinenin kendisi değil de bir parçası o fabrikada üretilmiş olabileceği için, öğenin alt parçaları da aynı `for` döngüsünde aranır; iki ayrı döngü yazmaktan daha mantıklıdır. `getParts()` null döndürürse öğe ya bir parçadır ya da alt parça tutulmamıştır, bir şey yapılmaz. Hiç öğe bulunmazsa sonunda “No such item found.” yazılır.

Ana program (ekrandan, 01:50:00):

```
public class MainApp {
    public static void main(String[] args) {
        ManufacturingItem[ ] items = new ManufacturingItem[4];
        items[0] = new MachinePart(99, 127, "Krank şaftı KŞ-7", "Bursa");
        Machine jenerator = new Machine(135, "Jeneratör JMX-99", "İstanbul");
        jenerator.addPart(new MachinePart(259,135,"Flanş FTR-5","Bursa"));
        jenerator.addPart(new MachinePart(378,135,"Kasnak KS-9","İstanbul"));
        jenerator.addPart(new MachinePart(196,135,"Rulman RL-3","Ankara"));
        items[1] = jenerator;
        items[2] = new MachinePart(63, 76, "Distribütör DST-12", "Bursa");
        items[3] = new MachinePart(82, 19, "Pnömatik Vana VPN-7", "Ankara");
        int sayi = Report.findItems(items, "Bursa");
        System.out.println("\t" + sayi + " parça bulundu.");
    }
}
```

Bir makineye üç parça eklenir ama parçalar kataloğa ayrıca eklenmez; kod buna rağmen onları bulabilmelidir. Derste çalıştırıldığında üç parça bulundu: krank şaftı, flanş ve distribütör (01:53:07). Ayrıntı için Örnek Sorular bölümüne bakın.

6.5 Kod kusuru: duplicate code

Hoca, refactoring konuları görüldükten sonra gözlere çarpması gereken bir sorun gösterdi: iki `if` bloğundaki yazdırma kodu birbirinin tamamen aynıdır. Bu bir “duplicate code” kod ku-

surudur. Örneğin listeleme biçimi değişip sekme yerine virgülle ayırmak ya da parça adlarını hiç yazmamak istenirse iki yerde değişiklik yapmak gerekir. Kopyala-yapıştır programlamanın sakıncası budur. Doğrusu bu ortak kodu ayrı bir static metoda taşımaktır; hoca bunun Extract Method ile yapılacağını not aldı.

7. Decorator Kalıbı

7.1 Amaç

Decorator kalıbının amacı nesnelere çalışma anında, yani dinamik olarak ek sorumluluklar atamaktır. Hoca bunun iddialı bir cümle olduğunu, ama gerekli altyapı kurulunca yerine getirilebileceğini söyledi. Normalde bu iş ancak kodlama anında yapılabilir: yeni bir sorumluluk için metot eklenir ya da kalıtımla yeni bir alt sınıf oluşturulur ve o sorumluluğu taşıması gereken nesneler bu alt sınıftan tanımlanır. Kalıtımda özelleşme sınıf düzeyindedir, yani o türden bütün nesneler için geçerlidir. Decorator ise nesnelerin bireysel olarak, aynı türden diğer nesnelerden farklı biçimde ve çalışma anında özelleşmesini sağlar.

7.2 Örnek ve alternatif çözümler

Yine bir GUI çerçevesi yazılıyor. Birkaç satır yazılabilen dikdörtgen bir metin alanı vardır. Bazı metin alanlarına çerçeve (border) çizilmek, bazılarına yatay, bazılarına dikey kaydırma çubuğu, bazılarına hepsi eklenmek istenir. Hoca, Dörtlü Çete'nin önerisinden önce iki alternatifi tartıştı.

Çözüm 1: her birleşim için alt sınıf. `TextArea`'dan `HorizontalScrollTextArea`, `VerticalScrollTextArea`, `BothScrollTextArea`, `BorderTextArea` gibi alt sınıflar türetilir. Bunların birleşimleri de istenir (hem kaydırma hem çerçeve), üç ayrı seçenek için çok sayıda birleşim ve çok sayıda sınıf ortaya çıkar. Daha önemli sorun şudur: kaydırma yetenekleri bir sınıfta, çerçeve yetenekleri başka bir sınıfta kodlanır; hem kaydırma hem çerçeve gerekiyorsa ikisinin kodu `ScrollBorderTextArea`'da yeniden yazılır. Bu, az önce küçük ölçekte görülen duplicate code tuzağına daha büyük ölçekte düşmektir. Aynı yetenekler paneller için de istenirse bir dolu sınıf ve bir dolu kopyalanmış kod ortaya çıkar; hocanın deyişiyle "tam bir bakım kabusu".

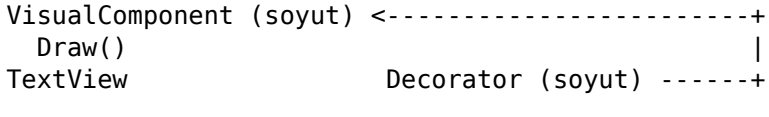
Çözüm 2: çoklu kalıtım. Yalnız metin alanları için düşünüldüğünü varsayalım. `TextArea`'dan kalıtımla `ScrollTextArea` ve `BorderTextArea`, bu ikisinden çoklu kalıtımla `ScrollBorderTextArea` türetilebilir (ekrandan, 02:10:00). Slayttaki zayıf yönler:

- Çoklu kalıtım her dilde desteklenmez.
- Çoklu kalıtımın bu çözümdeki şekli "diamond problem" sorununun bir örneğidir.
- `TextArea`'ya ek olarak `Panel` sınıfına kaydırma ve çerçeve yeteneği kazandırmak için ek üç sınıf daha gerekecektir. Üstelik bu durumda kaydırma ve çerçeve yeteneklerinin tek yerde gerçeklenmesi avantajı ortadan kalkacaktır.

Sınıflar birer nokta gibi düşünülüp birleştirildiğinde bir elmas şekli oluştuğu için buna "diamond inheritance" denir. Sorun şudur: üst sınıftaki bir metot iki ara sınıfta da override edilmişse ve en alttaki sınıfta override edilmemişse, hangi ebeveynden gelen metot geçerli olacaktır? Diller bunu çeşitli kurallarla çözebilir: örneğin Java destekleseydi `extends ScrollTextArea, BorderTextArea` yazıldığında listede ilk (ya da son) sıradaki ebeveynin metodu geçerli sayılabilir, ya da böyle bir metodun alt sınıfta override edilmesi şart koşulabilir. Bu ciddi bir karmaşıklık olarak değerlendirildiği için Java ve C# gibi güncel dillerde çoklu kalıtım desteklenmez; onun yerine arayüz (interface) kullanılır.

7.3 Dörtlü Çete'nin çözümü

Hoca, herhangi bir zayıf noktası olduğunu düşünmediği ilginç bir kalıp olarak Dörtlü Çete'nin çözümüne geçti. Slayttaki ilke: "Sınıfları değil de nesneleri birbirleri ile ilişkilendirelim." Özelleşme nesne düzeyinde istendiğine göre nesneler birbiriyle ilişkilendirilir ve sınıf tasarımı ona göre kurulur (ekrandan, 02:20:00):



```
Draw()                    Draw()   // component->Draw()
                        ScrollDecorator          BorderDecorator
                          Draw()                   Draw()   // Decorator::Draw(); DrawBorder
                          ScrollTo()               DrawBorder()
                          scrollPosition           borderWidth
```

```
Nesne zinciri: aBorderDecorator --component--> aScrollDecorator --component--> aTextView
```

GUI çerçevesinde soyut bir `VisualComponent` üst sınıfı vardır; `TextView`, `Panel` gibi kaydırılması ya da çerçevelenmesi gerekebilecek bütün görsel bileşenler onun alt sınıflarıdır. Yeni bir sınıf olarak soyut `Decorator` eklenir. Her farklı yetenek `Decorator`'ın bir alt sınıfı olarak kodlanır: kaydırma yeteneği kazandıran `ScrollDecorator`, çerçeve yeteneği kazandıran `BorderDecorator`. Yatay ve dikey kaydırma ayrı istenirse `HorizontalScrollDecorator` ve `VerticalScrollDecorator` diye iki sınıf konur. Bir `TextView`'a yalnız kaydırma eklenecekse yalnız bir `ScrollDecorator` ile, hem kaydırma hem çerçeve eklenecekse hem `BorderDecorator` hem `ScrollDecorator` ile ilişkilendirilir.

Hoca şemadaki ilişki gösterimini eleştirdi: soldaki nesne şemasında bir dekoratör tek bir nesneyle ilişkilidir, bu yüzden `Decorator` ile `component` arasındaki ilişki birebirdir. Eski gösterimdeki elmas genelde karşı uçta "yıldız" (çok) anlamına gelir ve yanıltıcıdır; bu ilişki düz çizgiyle ve birebir olarak çizilmelidir.

Kalıbı mümkün kılan altyapı `Draw` metotlarındadır. `Decorator` soyut bir sınıf olmakla birlikte `Draw()` metodu soyut değildir: ilişkili olduğu bileşenin `Draw()` metodunu yürütür. Somut alt sınıflarda, örneğin `BorderDecorator.Draw()` önce ilişkili nesnenin kendini çizmesini sağlar, sonra `DrawBorder()` ile çerçeveyi çizer. `ScrollDecorator` da `Draw()` içinde önce ilişkili bileşenin çizimini çağırır ve ayrıca `ScrollTo()` gibi kaydırma yeteneklerine sahiptir. Bir `Panel` de bir `ScrollDecorator`'a verilerek kaydırma ya da çerçeve yeteneği kazanabilir.

Hoca iki gerçekleme seçeneğinden söz etti: `Decorator.Draw()` somut kalır ve alt sınıflar onu çağırır; ya da `Decorator.Draw()` soyut yapılır ve her alt sınıf `component.Draw()` çağrısını kendisi yazar. İkisi de aynı yere götürür. İlk seçeneğin küçük bir kazancı vardır: çağrı üst sınıfta yazıldığı için alt sınıfta unutulamaz. Hoca kendi örneklerinde dinamik çağrıyı (static olmayan) tercih ettiğini söyledi.

> **Ders sonrası not:** Hoca slayttaki `Decorator::Draw();` gösterimine bakarak metodun static olduğunu söyledi. C++'ta `Decorator::Draw()` biçimi, alt sınıf içinden üst sınıfın aynı adlı metodunu çağırmak için kullanılır; burada büyük olasılıkla Java'daki `super.draw()` anlamındadır ve metodun static olduğunu göstermez.

7.4 Kalıp yapısı

Kalıp yapısında soyut bir `Component`, ondan türeyen somut `ConcreteComponent`'ler ve ek yetenek kazandırmayı sağlayan soyut bir ara sınıf `Decorator` vardır. `Decorator`'ın `Operation()` metodu somut olabilir ya da soyut bırakılıp alt sınıflarda gerçeklenebilir. Her `ConcreteDecorator` ilişkili olduğu bileşene farklı bir yetenek kazandırır: ya önce ilişkili bileşenin işlemi çağrılır sonra ek yetenek yürütülür, ya da önce ek yetenek yürütülür sonra ilişkili bileşen çağrılır.

7.5 Ne zaman kullanılır

Slayttaki maddeler ve hocanın açıklamaları (ekrandan, 02:30:00):

- **Tek tek nesnelere dinamik olarak (çalışma anında) yeni sorumluluklar eklenmek istendiğinde.**
- **Tek tek nesnelere şeffaf olarak (diğerlerini etkilemeden) yeni sorumluluklar eklenmek istendiğinde.** Belirli bir `TextView`'a eklenen yetenek diğer `TextView`'lara eklenmiş olmaz.
- **Bazı sorumlulukların iptal edilebileceği veya askıya alınabileceği durumlarda.** Hocanın örneği: bir `Doctor` sınıfının tedavi etme metodu vardır, ama doktor hastayken tedavi yapmamalıdır. Bu bir `isIll` alanıyla modellenebilir; ama doktor bir "hasta olabilir" dekoratörüyle ilişkilendirilirse, dekoratör ilgili işlemi çağırmadan önce denetim yapıp işi yürütmeyebilir. Hoca bunun yalnız Decorator'a özgü olmadığını, Proxy ile de yapılabileceğini, ne kadar mantıklı olduğunun probleme göre değiştiğini söyledi.

- **Kalıtımın uygun olmadığı anlarda.** Örneğin sınıf kodu mevcut değilse ya da yeni eklemeler üstel artan sınıf sayısına neden olacaksa. Ayrıca bir sınıfın yalnız bir üst sınıfı olabildiğinden, bu tek kalıtım hakkını harcamak istenmiyorsa Decorator akıllıca kullanılabilir.
- **Java'nın stream tabanlı I/O kütüphanesi bu kalıbı kullanır.** Hoca Java'nın Swing çerçevesini de örnek verdi.

7.6 Zayıf yön ve dikkat edilecek nokta

Slayta göre yeni sorumluluk ancak şu biçimlerde eklenebilir:

- Eski sorumluluğun tamamen iptal edilmesi. Dikkat: bu durumda nesne zinciri kopabilir.
- Eski sorumluluktan önce ve/veya sonra yeni komutlar eklenmesi.

Hoca bunu bir işçi örneğiyle açıkladı: bir işçi bir ürün üretiyorsa, ürettikten sonra üstünü boyamak ya da ürünü başka bir ürünle birleştirmek eklenebilir; ama ürünün iç ayrıntıları ve çalışma biçimi değiştirilemez. Mevcut yeteneği değiştirmenin tek yolu, ilgili dekoratörde ya da sınıfın kendisinde onu silip yeniden yazmaktır.

Hocanın sonradan eklediği önemli bir ayrıntı: yeni yeteneklere doğrudan dekore edilen nesne üzerinden değil, en üstteki dekore eden nesne üzerinden erişilmelidir. İstek doğrudan bileşene gönderilirse dekorasyonlar devreye girmez; en öndeki dekoratöre gönderilen istek zincir boyunca diğer ek sorumlulukların da yürütülmesini sağlar.

Hoca burada kalıp adlarını neden çevirmediğini de açıkladı: Türkçe karşılıkların taraftarı olsa da tasarım kalıplarının önemli bir özelliği yazılımcılar arasında ortak bir dil ve sözlük oluşturmaktır; bu yüzden "Decorator"ı "süsleyici" diye çevirmiyor, "dekore eden" ve "dekore edilen" demeyi tercih ediyor.

8. Decorator Örneği: Satış Makbuzu

8.1 Problem ve adım adım gelen istekler

Bir e-ticaret yazılımına satış emirlerinin makbuzlarını basma işlevi eklenecektir. Zaten bir `TaskController` ve satışları simgeleyen `SalesOrder` sınıfı vardır. `SalesOrder`'ın `process` metoduna ek olarak bir `printTicket` metodu eklenir. Makbuzun kendine özgü bir biçimi (başlık, ara satırlar vb.) olacağı için, bu iş ayrı bir `SalesTicket` sınıfına verilir; `SalesOrder.printTicket()` içinde `mySalesTicket.printTicket()` çağrılır.

Hoca kitaptaki şemayı eleştirdi: `mySalesTicket` nesnesinin nereden geldiği belli değildir. Parametre olarak gelebilir, ama ona göre daha doğru modelleme, `SalesOrder` ile `SalesTicket` arasında üye alan üzerinden düz çizgiyle gösterilen bir sahiplik ilişkisidir.

Yeni istek: makbuzlara üst bilgi ve alt bilgi eklenecektir. Bunların da kendi biçimleri olacağı için `Header` (`printHeader`) ve `Footer` (`printFooter`) sınıfları yazılır. Müşteri her makbuzda üst ya da alt bilgi olması gerekmediğini söyler; `printTicket`'a boolean parametreler verilebilir ya da metot aşırı yüklenebilir (parametresiz sürüm ikisini de basar). Biraz kolaya kaçmaktır ve if'ler gerekir, ama sorun değildir.

Sonra farklı türde üst ve alt bilgiler gelirse kalıtım kullanılır: `Header` ve `Footer` soyut yapılır (varsayılan bir tür varsa soyut olmaları gerekmez) ve `HeaderType1`, `HeaderType2`, `FooterType1`, `FooterType2` alt sınıfları oluşturulur (ekrandan, 02:40:00). Slayta göre bu çözüm, henüz incelenmemiş olan Strategy kalıbının bir gerçeklemesidir. Hoca burada da `SalesTicket` ile `Header/Footer` arasındaki elmas gösterimini eleştirdi ve düz çizgiye çevirdi.

Son istek: aynı makbuzda birden fazla farklı üst bilgi ve/veya alt bilgi olabilir; bunlardan biri hiç olmayabilir. Slayttaki örnekler:

```
HEADER1      HEADER1
HEADER2      Receipt
Receipt      FOOTER1
FOOTER2      FOOTER2
```

Bu noktada hoca elmasın gerekçesini daha iyi anladığını, birden çok üst ve alt bilgi için ilişkinin bire çok olacağını ve ekleme-silme metotlarının gerekeceğini söyledi. Ama slayta göre

bu durumda gereksinimi ancak Decorator kalıbı karşılar. Hoca, müşteri bu istekleri baştan söyleseydi tasarımı en başında Decorator ile yapacağını, çünkü hangi üst ve alt bilgilerin ekleneceğinin makbuzu "süslemek" olduğunu söyledi.

8.2 Decorator ile çözüm

Decorator altyapısını kullanmak için `SalesTicket`'a soyut bir üst sınıf (`Component`) eklenir. `TicketDecorator` ara sınıfı yazılır ve farklı üst bilgi ve alt bilgi türleri onun alt sınıfları olur. Header sınıflarında üst bilgiyi basan, Footer sınıflarında alt bilgiyi basan metotlar vardır; bunlar yalnız aynı sınıftaki `printTicket` tarafından çağrılacağı için private olabilir. Bir header önce kendi üst bilgisini basar, sonra dekore ettiği bileşenin `printTicket` metodunu çağırır. Bir header doğrudan `SalesTicket` ile değil, başka bir header ya da footer ile ilişkili olabileceği için bu yapı gereklidir. Footer ise önce makbuzu bastırır, sonra alt bilgiyi basar. Bu örnek, farklı dekoratörlerin asıl sorumluluktan önce ya da sonra devreye girebildiğini gösterir.

Hoca `TicketDecorator.printTicket()` içinde bileşenin null olup olmadığının da denetlenebileceğini, Dörtlü Çete'nin örneğinde bu düşünülmemiş olsa da eklenebileceğini söyledi.

8.3 Kod

Kodda her sınıf, hangi anda hangi nesnenin metodunun devreye girdiğini göstermek için kendi sınıf adını ekrana yazar. `Component`, `printTicket()` metodunu soyut tanımlar. `SalesTicket` nesnesi çağrıldığında ekrana paket adıyla birlikte `SalesTicket` yazılır. `TicketDecorator`'da bileşen kurucuda mutlaka atanır; `prtTicket()` ise bileşen null değilse bileşenin `prtTicket()` metodunu çağırır.

`Header1` ve `Header2` (ekrandan, 02:46:30):

```
package decorator.example;
public class Header1 extends TicketDecorator {
    public Header1 (Component myComponent) {
        super(myComponent);
    }
    public void prtTicket() {
        // place printing header 1 code here
        System.out.println(getClass().getName());
        super.prtTicket();
    }
}
public class Header2 extends TicketDecorator {
    public Header2 (Component myComponent) {
        super(myComponent);
    }
    public void prtTicket () {
        // place printing header 2 code here
        System.out.println(getClass().getName());
        super.prtTicket();
    }
}
```

Footer'lar benzerdir, yalnız önce `super.prtTicket()` ile ilgili bileşenin makbuzunu yazdırır, sonra kendini yazar. Hoca kodda hiç static metot olmadığını, yalnız soyut ya da somut nesne metotları bulunduğunu vurguladı.

İstemci ve fabrika (ekrandan, 02:50:00):

```
package decorator.example;
public class Client {
    public static void main(String[] args) {
        Factory myFactory = new Factory();
        Component myComponent = myFactory.getComponent();
        myComponent.prtTicket();
    }
```

```
}
public class Factory {
    public Component getComponent () {
        return(new Header1( new Header2( new Footer2(
                new Footer1( new SalesTicket()))))));
    }
}
```

Hangi üst ve alt bilgi birleşiminin seçileceğine (dekorasyona) Client karar verir. Slayta göre tasarım bu aşamada Factory Method kalıbıyla geliştirilebilir; hoca iki kalıp birden kurup örneği karmaşıklaştırmamak için yalnız basit bir `Factory` sınıfı eklediğini, Factory Method'un da Abstract Factory gibi çok sınıf eklemediğini söyledi. Son tasarımda `Factory`, soyut `Component`'e bağımlıdır; tasarımın geri kalanı aynıdır.

Kurucuların iç içe yazılmasına zincirleme (chaining) denir. Zincirleme kullanılmadan nesneler tek tek de ilişkilendirilebilir. Dekoratöre dekore edeceği varlık verilmek zorundadır ve bu en iyi kurucuyla yapılır. Hoca, Dörtlü Çete'nin bunun kurucuyla mı ayrı bir metotla mı yapılacağını söylemediğini, ama dekoratörün işleminde null denetimi olmadığı için büyük olasılıkla kurucuyu öngördüğünü belirtti.

8.4 Çalıştırma ve sıralama

Hoca önce zincirlemeyi yorum satırı yapıp nesneleri tek tek ilişkilendirdi: bir `Component` işaretçisine `new SalesTicket()` atanır, sonra aynı işaretçiye bu nesneyi saran `new Header2(myComponent)` atanır, ve böyle devam eder. Ayrı bir dekoratör işaretçisi tanımlamaya gerek yoktur, çünkü üst sınıftan bir nesnenin beklendiği her yerde alt sınıftan bir nesne kullanılabilir. `new` her seferinde yeni bir nesne döndürür; önceki nesne silinmez, bellekte yaşamaya devam eder, yalnız işaretçi yeniden kullanılır. Hoca `return myComponent;` yazmayı unuttuğunu fark edip ekledi.

Çıktıyı belirleyen kural şudur: header önce kendini yazar sonra iletir; footer önce iletir sonra kendini yazar. Bu yüzden aynı türden birden çok dekoratör kullanılıyor ve kendi aralarındaki sıra önemliyse ilişkilendirme sırasına dikkat edilmelidir. Header'lar için dıştaki header önce yazılır; footer'lar için ise makbuza (iç tarafa) en yakın footer önce yazılır. Hocanın tek tek ilişkilendirdiği ilk denemede çıktı `Header1, Header2, SalesTicket, Footer2, Footer1` oldu; footer sırasını değiştirmek için ilişkilendirme sırası değiştirilmelidir. İç içe yazılan sürümde de ilk denemede footer'lar ters sırada çıktı, hoca `Footer1` ile `Footer2`'nin yerini değiştirerek istenen sırayı elde etti. Bir header'a ihtiyaç yoksa ilgili satır yorum yapılır ve o dekorasyon eklenmez; iç içe yazımda bir dekoratör çıkarılınca bir kapanış parantezinin de silinmesi gerekir (bunu derste bir öğrenci hatırlattı).

Slayttaki çalışma sonucu (ekrandan, 03:00:00):

```
HEADER1
HEADER2
Receipt
FOOTER1
FOOTER2
```

Hoca bu çözümün epey esnek olduğunu söyledi.

8.5 Java stream'lerinde zincirleme

Dersin sonunda hoca, kurucuları iç içe yazarak yapılan zincirlemenin Java'nın stream kütüphanesinde kullanıldığını hatırlattı: bir dosyayı sıkıştırmak ya da şifrelemek için ilgili stream kurucuları birbirine zincirlenir ve istenen yetenekler bu şekilde birleştirilir.

> **Ek bilgi (derste anlatılmadı):** Java'da örneğin `new BufferedReader(new InputStreamReader(new FileInputStream("a.txt")))` ya da `new GZIPOutputStream(new FileOutputStream("a.gz"))` yazımları Decorator kalıbının tipik kullanımlarıdır. Her sarmalayıcı sınıf aynı soyut türden (`Reader`, `OutputStream`) türer, sardığı akışa iletir ve tamponlama, sıkıştırma gibi bir yetenek ekler.

Örnek Sorular

1. (derste sözlü, 00:46:11) Bridge örneğinin ilk çözümünde iki şekil (dikdörtgen, daire) ve iki çizim ailesi (DP1, DP2) varken kaç somut şekil sınıfı gerekir? Üçüncü bir şekil eklenirse ne olur? Bridge ile bu sayı nasıl değişir?

> **Çözüm (derste verildi):** İlk çözümde sınıf sayısı şekil sayısı ile aile sayısının çarpımıdır: $2 \times 2 = 4$ (`V1Rectangle`, `V2Rectangle`, `V1Circle`, `V2Circle`). Üçüncü şekille $2 \times 3 = 6$ olur. Bridge ile şekiller ve çizim yöntemleri ayrı hiyerarşilerde olduğu için her yeni şekil için yalnız bir sınıf (örneğin `Triangle`), her yeni çizim ailesi için yalnız bir `Drawing` alt sınıfı eklenir.

2. (ekrandan, 00:40:00; derste sözlü, 01:09:49) Bridge çözümünde `drawLine` ve `drawCircle` neden protected yapılmıştır? Bu zorunlu mudur?

> **Çözüm (derste verildi):** Slayttaki gerekçe, bu metotların nesnenin iç çalışmasıyla ilgili olduğu, private metotların kalıtımla aktarılmadığı ve protected metotların başka türden nesnelere görünmeyip alt sınıflara aktarıldığıdır. Hocaya göre zorunlu değildir: `Shape` soyut olduğundan ondan nesne oluşturulup metot dışarıdan çağrılamaz, metot static olmadığından sınıf adıyla da çağrılamaz. Alt sınıfların köprü üzerinden yapılan çizimi override ederek bozması engellenmek isteniyorsa metot `public final` yapılabilir.

3. (derste sözlü, 01:30:28) Composite kalıbında `Leaf` sınıfı `Add`, `Remove`, `GetChild` metotlarını nasıl gerçekler? Bu neden kalıbın zayıf yönüdür?

> **Çözüm (derste verildi):** Yaprağın çocuğu olamayacağı için bu metotlar onun için anlamsızdır, ama soyut `Component`'ten kalıtımla geldikleri için derleme hatası almamak adına yazılmak zorundadır. Gövdeleri boş bırakılır; boolean döndüren `Add` için `return false` (ya da şikayet etmeden geçmesi istenirse `return true`), `GetChild(int)` için `return null` yazılır. Alt sınıfa anlamsız metotların kalıtımla geçmesi kalıtım ilişkisinin özüne aykırıdır; bu, kalıbın esnekliği için ödenen bedeldir.

4. (ekrandan, 01:50:00; derste çalıştırıldı, 01:53:07) Makine kataloğu örneğinde `Report.findItems(items, "Bursa")` çağrısı ne yazdırır ve kaç döndürür?

> **Çözüm (derste verildi):** Kod hem katalogdaki öğelere hem de makinenin parçalarına baktığı için üç öğe bulur: katalogdaki krank şaftı, jeneratörün parçası flanş ve katalogdaki distribütör. Derste de üç parça bulundu. Başlık satırlarından sonra kayıtlar şöyledir (python ile doğrulandı):
>
> ```
> 1.  99   Krank şaftı KŞ-7
> 2.  259  Flanş FTR-5
> 3.  63   Distribütör DST-12
>     3 parça bulundu.
> ```
>
> Jeneratörün kendisi İstanbul'da, kasnak İstanbul'da, rulman ve pnömatik vana Ankara'da üretildiği için listelenmez. Flanş kataloğa ayrıca eklenmediği halde `getParts()` üzerinden bulunur.

5. (ekrandan, 01:48:00; derste sözlü, 01:50:26) `Report.findItems` kodundaki tasarım sorunu nedir ve nasıl giderilir?

> **Çözüm (derste verildi):** Öğe ve alt parça için yazılan iki `if` bloğundaki sayaç artırma ve yazdırma kodu birbirinin aynıdır; bu "duplicate code" kod kusurudur. Listeleme biçimi değişirse iki yerin de değiştirilmesi gerekir. Ortak kod Extract Method ile ayrı bir static metoda taşınmalıdır.

6. (ekrandan, 02:10:00; derste sözlü, 02:10:17) `ScrollTextArea` ve `BorderTextArea`'dan çoklu kalıtımla `ScrollBorderTextArea` türetmenin sorunları nelerdir?

> **Çözüm (derste verildi):** Çoklu kalıtım her dilde desteklenmez (Java ve C# desteklemez). Yapı "diamond problem" örneğidir: üst sınıftaki bir metot iki ara sınıfta farklı biçimde override edilmişse en alttaki sınıfın hangisini alacağı belirsizdir. Ayrıca aynı yetenekler `Panel`'e de kazandırılmak istenirse üç ek sınıf daha gerekir ve kaydırma ile çerçeve kodunun tek yerde gerçeklenmesi avantajı kaybolur.

7. (ekrandan, 02:50:00 ve 03:00:00) Header'ların önce kendini yazıp sonra ilettiği, Footer'ların önce iletip sonra kendini yazdığı varsayımıyla `new Header1(new Header2(new Footer2(new Footer1(new SalesTicket()))))` zincirinde `prtTicket()` çağrılınca çıktı ne olur?

> **Çözüm (derste verildi):** Slayttaki çalışma sonucu `HEADER1, HEADER2, Receipt, FOOTER1, FOOTER2`dir. `Header1` kendini yazar ve `Header2`'ye iletir; `Header2` kendini yazar ve `Footer2`'ye iletir; `Footer2` önce `Footer1`'e iletir; `Footer1` önce `SalesTicket`'a iletir ve makbuz yazılır, sonra `Footer1` kendini yazar, en son `Footer2` kendini yazar. Bu akış python ile de doğrulandı. Footer'lar için makbuza en yakın olan önce yazılır; sırayı değiştirmek için ilişkilendirme sırası değiştirilmelidir.

Dersin Toparlaması

Bu derste yaratımsal kalıplardan yapısal kalıplara geçildi ve dört yapısal kalıp işlendi. Adapter, imzaları uyuşmayan bir sınıfı, istekçinin beklediği imzayı taşıyan ve çağrıyı ileten bir ara sınıfla kullanılabilir kılar; çizim programına hazır `XXCircle` sınıfının `Circle` adaptörüyle uydurulması bunu gösterdi. Bridge, soyutlama ile gerçeklemeyi ayrı hiyerarşilere ayırıp bir üye alanla bağlar; pencere ve iki çizim kütüphanesi örneklerinde, tek hiyerarşide çarpımla artan sınıf sayısının her yeni şekil ya da yöntem için tek sınıfa indiği görüldü. Erişim belirleyici tartışmasında protected yerine `public final` kullanılabileceği gösterildi.

Composite, hiyerarşik parça-bütün ilişkilerini modelleyip tek nesnelerle toplulukların aynı biçimde kullanılmasını sağlar; üst düğüm başvurusu, `getRoot()`, ara düğüm silme gibi tasarım kararları ve Leaf'e anlamsız çocuk yönetim metotlarının geçmesi zayıflığı tartışıldı, makine kataloğu örneğinde duplicate code kusuru da işaretlendi. Decorator ise sınıfları değil nesneleri ilişkilendirerek tek tek nesnelere çalışma anında yeni sorumluluk ekler; metin alanı ve satış makbuzu örneklerinde çoklu kalıtımın diamond problemi, zincirleme kurucular ve dekoratör sırasının çıktıya etkisi gösterildi. Hoca derste Facade ve Flyweight'ın sırada olduğunu söyledi; bir sonraki derste yapısal kalıplara büyük olasılıkla Facade ile devam edilecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.